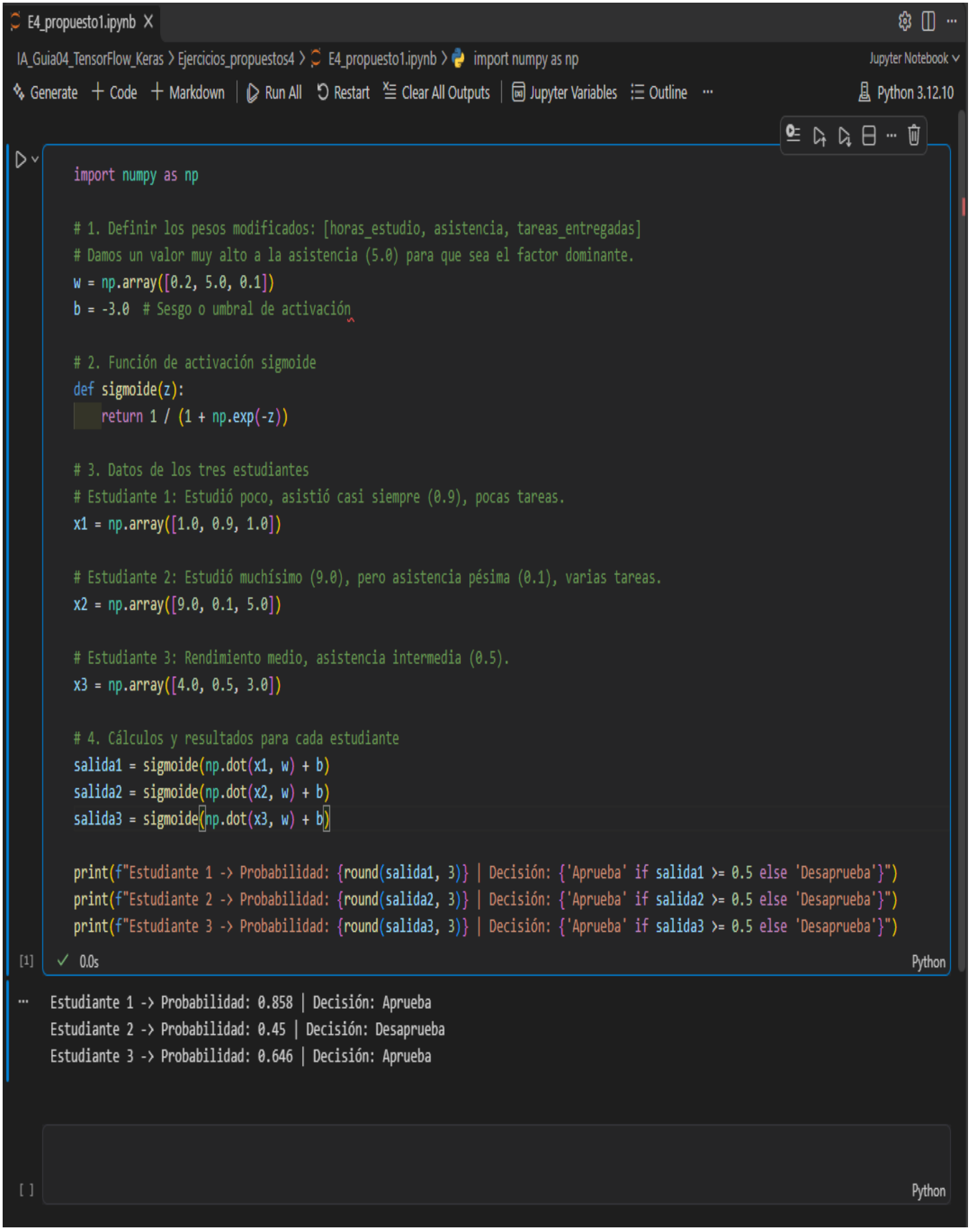
1. En 01_neurona.py, cambia los pesos para que la asistencia sea lo más importante. Prueba con tres estudiantes distintos y explica los resultados.



2. En 02_activaciones.py, agrega una quinta gráfica con la función lineal (y = z).

E4_propuesto2.ipynb

IA_Guia04_TensorFlow_Keras > Ejercicios_propuestos4 > E4_propuesto2.ipynb > import numpy as np

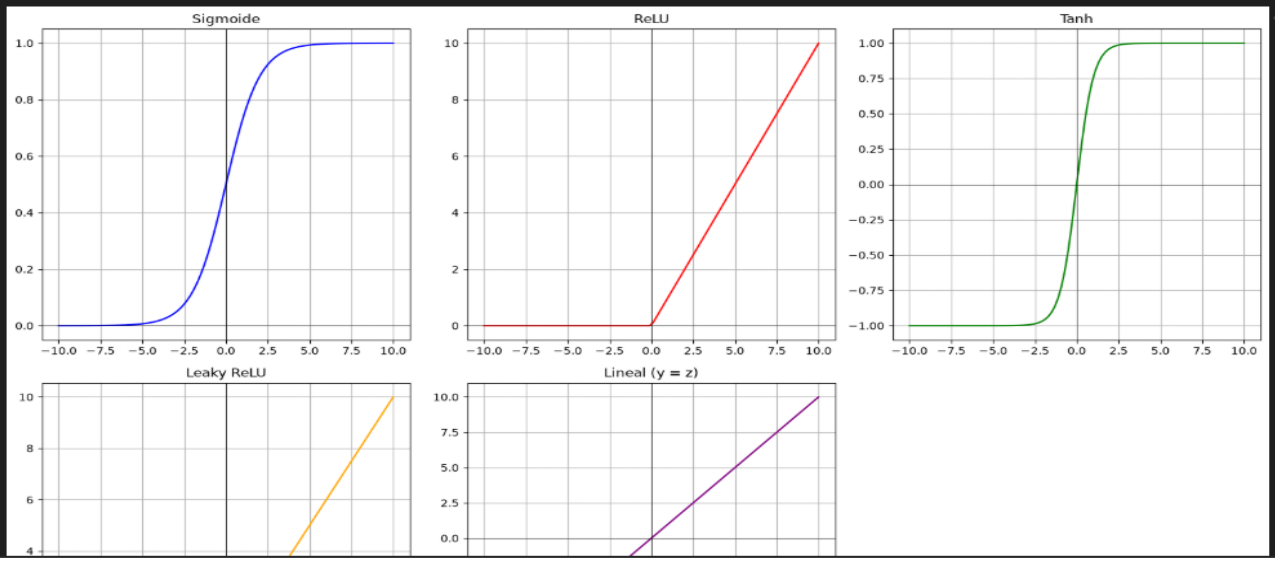
Jupyter Notebook

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | ...

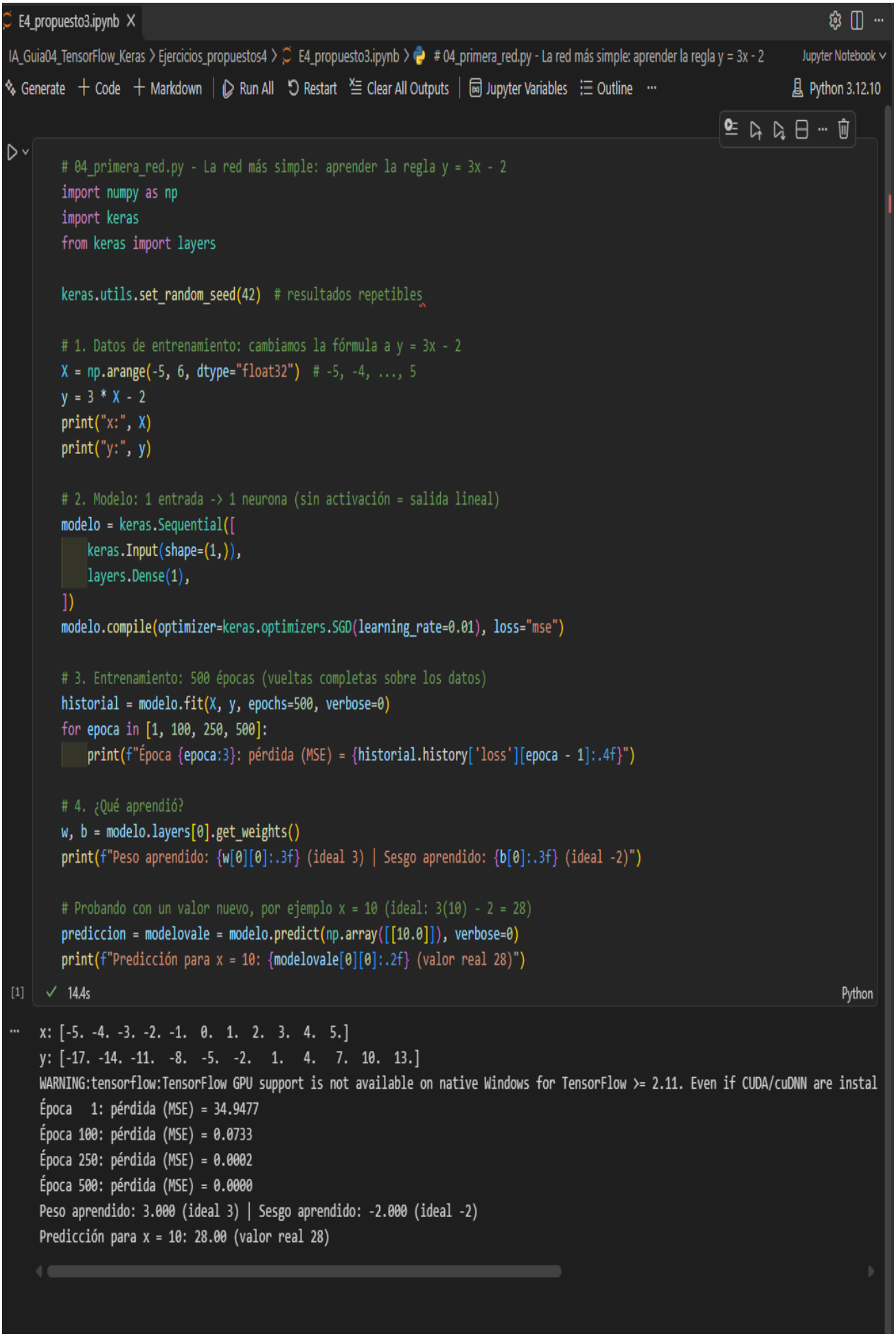
Python 3.12.1

```python
import numpy as np
import matplotlib.pyplot as plt
# 1. Definir el rango de valores para z (eje X)
z = np.linspace(-10, 10, 100)
# 2. Definir las funciones de activación
def sigmoide(z):
    return 1 / (1 + np.exp(-z))
def relu(z):
    return np.maximum(0, z)
def tanh(z):
    return np.tanh(z)
def leaky_relu(z, alpha=0.1):
    return np.where(z > 0, z, z * alpha)
# 5. Función Lineal (identidad: y = z)
def lineal(z):
    return z
# 3. Calcular los valores de salida para cada función
y_sigmoide = sigmoide(z)
y_relu = relu(z)
y_tanh = tanh(z)
y_leaky = leaky_relu(z)
y_lineal = lineal(z)
# 4. Crear la figura y las subgráfi-cas (grid de 2x3 o una fila grande)
plt.figure(figsize=(15, 10))
# Gráfica 1: Sigmoide
plt.subplot(2, 3, 1)
plt.plot(z, y_sigmoide, color='blue')
plt.title('Sigmoide')
plt.grid(True)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
# Gráfica 2: ReLU
plt.subplot(2, 3, 2)
plt.plot(z, y_relu, color='red')
plt.title('ReLU')
plt.grid(True)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
# Gráfica 3: Tanh
plt.subplot(2, 3, 3)
plt.plot(z, y_tanh, color='green')
plt.title('Tanh')
plt.grid(True)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
# Gráfica 4: Leaky ReLU
plt.subplot(2, 3, 4)
plt.plot(z, y_leaky, color='orange')
plt.title('Leaky ReLU')
plt.grid(True)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
# Gráfica 5: Lineal (Nueva)
plt.subplot(2, 3, 5)
plt.plot(z, y_lineal, color='purple')
plt.title('Lineal (y = z)')
plt.grid(True)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
# Ajustar diseño y mostrar
plt.tight_layout()
plt.show()
```

[1] ✓ 0.5s Python



3. En 04_primera_red.py, cambia los datos para que la red aprenda y = 3x − 2. ¿Qué peso y sesgo aprendió?



4. En 05_red_iris.py, cambia la capa de 10 neuronas por una de 4 neuronas y luego por una de 32. Anota la exactitud en prueba de cada caso en una tabla.

E4_propuesto4.ipynb

IA_Guia04_TensorFlow_Keras > Ejercicios_propuestos4 > E4_propuesto4.ipynb > # 05_red_iris.py - Red neuronal multicapa con 32 neuronas

Jupyter Notebook

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | ... | Python 3.12.10

```python
# 05_red_iris.py - Red neuronal multicapa con 32 neuronas
import numpy as np
import matplotlib.pyplot as plt
import keras
from keras import layers
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
keras.utils.set_random_seed(42)
# 1. Datos: 150 flores, 4 medidas, 3 especies
iris = load_iris()
X, y = iris.data, iris.target
X_ent, X_pru, y_ent, y_pru = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
escalador = StandardScaler()
X_ent = escalador.fit_transform(X_ent)
X_pru = escalador.transform(X_pru)
print("Entrenamiento:", X_ent.shape, "| Prueba:", X_pru.shape)
# 2. Arquitectura: 4 entradas -> 32 (ReLU) -> 8 (ReLU) -> 3 (Softmax)
modelo = keras.Sequential([
    keras.Input(shape=(4,)),
    layers.Dense(32, activation="relu"),
    layers.Dense(8, activation="relu"),
    layers.Dense(3, activation="softmax"),   ])
modelo.summary()
# 3. Compilar: optimizador, función de pérdida y métrica
modelo.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
# 4. Entrenar
historial = modelo.fit(X_ent, y_ent, epochs=100, batch_size=8, validation_split=0.1, verbose=0)
print("Exactitud final en entrenamiento:", round(historial.history["accuracy"][-1], 3))
# 5. Evaluar con datos que la red nunca vio
perdida, exactitud = modelo.evaluate(X_pru, y_pru, verbose=0)
print(f"Exactitud en prueba: {exactitud:.3f}")
# 6. Predecir una flor nueva
flor = escalador.transform([[5.1, 3.5, 1.4, 0.2]])
prob = modelo.predict(flor, verbose=0)[0]
print("Probabilidades:", prob.round(3))
print("Especie predicha:", iris.target_names[np.argmax(prob)])
# 7. Guardar el modelo entrenado y la curva de aprendizaje
modelo.save("modelo_iris_32neuronas.keras")
print("Modelo guardado en modelo_iris_32neuronas.keras")
plt.plot(historial.history["accuracy"], label="Entrenamiento")
plt.plot(historial.history["val_accuracy"], label="Validación")
plt.title("Curva de aprendizaje (32 neuronas)")
plt.xlabel("Época")
plt.ylabel("Exactitud")
plt.legend()
plt.savefig("curva_aprendizaje_32.png", dpi=100, bbox_inches="tight")
plt.show()
```

[1] ✓ 9.1s Python

```
Entrenamiento: (120, 4) | Prueba: (30, 4)
```

```
Model: "sequential"
```

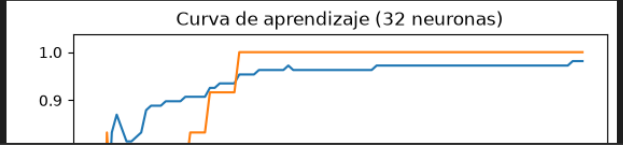
Layer (type)	Output Shape	Param #
dense (Dense)	(None, 32)	160
dense_1 (Dense)	(None, 8)	264
dense_2 (Dense)	(None, 3)	27

```
Total params: 451 (1.76 KB)

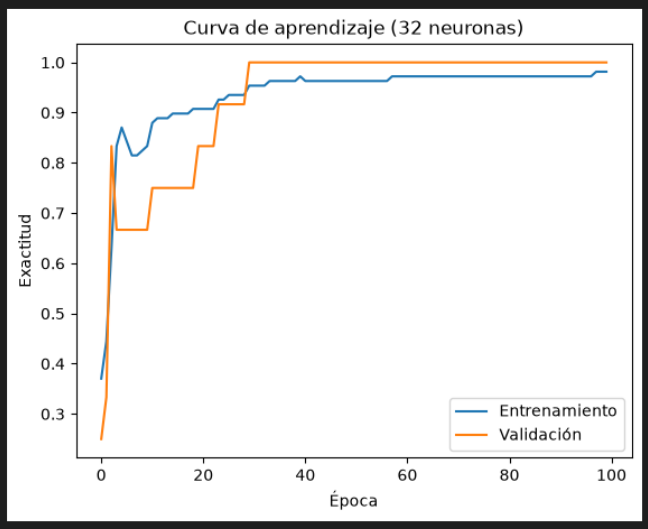
Trainable params: 451 (1.76 KB)

Non-trainable params: 0 (0.00 B)

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are instal
Exactitud final en entrenamiento: 0.981
Exactitud en prueba: 0.967
Probabilidades: [1. 0. 0.]
Especie predicha: setosa
Modelo guardado en modelo_iris_32neuronas.keras
```



```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are instal
Exactitud final en entrenamiento: 0.981
Exactitud en prueba: 0.967
Probabilidades: [1. 0. 0.]
Especie predicha: setosa
Modelo guardado en modelo_iris_32neuronas.keras
```



Python

```
Entrenamiento: (120, 4) | Prueba: (30, 4)
```

Model: "sequential"

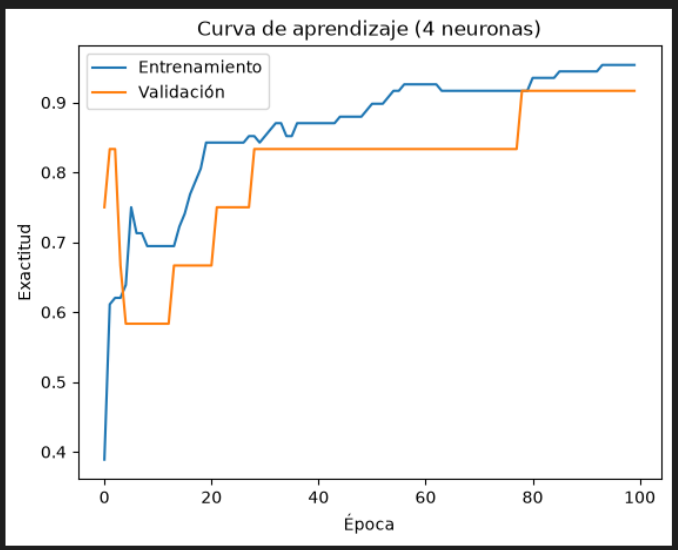
Layer (type)	Output Shape	Param #
dense (Dense)	(None, 4)	20
dense_1 (Dense)	(None, 8)	40
dense_2 (Dense)	(None, 3)	27

Total params: 87 (348.00 B)

Trainable params: 87 (348.00 B)

Non-trainable params: 0 (0.00 B)

```
Exactitud final en entrenamiento: 0.954
Exactitud en prueba: 0.933
Probabilidades: [0.999 0.    0.   ]
Especie predicha: setosa
Modelo guardado en modelo_iris_4neuronas.keras
```



5. Cambia epochs=100 por epochs=20. ¿Qué pasa con la exactitud y con la curva de aprendizaje?

E4_propuesto5.ipynb

IA_Guia04_TensorFlow_Keras > Ejercicios_propuestos4 > E4_propuesto5.ipynb > # 05_red_iris.py - Red neuronal multicapa con epochs=20 — Jupyter Notebook

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | ... — Python 3.12.10

```python
# 05_red_iris.py - Red neuronal multicapa con epochs=20
import numpy as np
import matplotlib.pyplot as plt
import keras
from keras import layers
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
keras.utils.set_random_seed(42)
# 1. Datos: 150 flores, 4 medidas, 3 especies
iris = load_iris()
X, y = iris.data, iris.target
X_ent, X_pru, y_ent, y_pru = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
escalador = StandardScaler()
X_ent = escalador.fit_transform(X_ent)
X_pru = escalador.transform(X_pru)
print("Entrenamiento:", X_ent.shape, "| Prueba:", X_pru.shape)
# 2. Arquitectura: 4 entradas -> 10 (ReLU) -> 8 (ReLU) -> 3 (Softmax)
modelo = keras.Sequential([
    keras.Input(shape=(4,)),
    layers.Dense(10, activation="relu"),
    layers.Dense(8, activation="relu"),
    layers.Dense(3, activation="softmax"),
])
modelo.summary()
# 3. Compilar: optimizador, función de pérdida y métrica
modelo.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
# 4. Entrenar con epochs=20
historial = modelo.fit(X_ent, y_ent, epochs=20, batch_size=8, validation_split=0.1, verbose=0)
print("Exactitud final en entrenamiento:", round(historial.history["accuracy"][-1], 3))
# 5. Evaluar con datos que la red nunca vio
perdida, exactitud = modelo.evaluate(X_pru, y_pru, verbose=0)
print(f"Exactitud en prueba: {exactitud:.3f}")
# 6. Predecir una flor nueva
flor = escalador.transform([[5.1, 3.5, 1.4, 0.2]])
prob = modelo.predict(flor, verbose=0)[0]
print("Probabilidades:", prob.round(3))
print("Especie predicha:", iris.target_names[np.argmax(prob)])
# 7. Guardar el modelo entrenado y la curva de aprendizaje (20 épocas)
modelo.save("modelo_iris_epochs20.keras")
print("Modelo guardado en modelo_iris_epochs20.keras")
plt.plot(historial.history["accuracy"], label="Entrenamiento")
plt.plot(historial.history["val_accuracy"], label="Validación")
plt.title("Curva de aprendizaje (20 épocas)")
plt.xlabel("Época")
plt.ylabel("Exactitud")
plt.legend()
plt.savefig("curva_aprendizaje_epochs20.png", dpi=100, bbox_inches="tight")
plt.show()
```

[1] Python

```
Entrenamiento: (120, 4) | Prueba: (30, 4)
```

```
Model: "sequential"
```

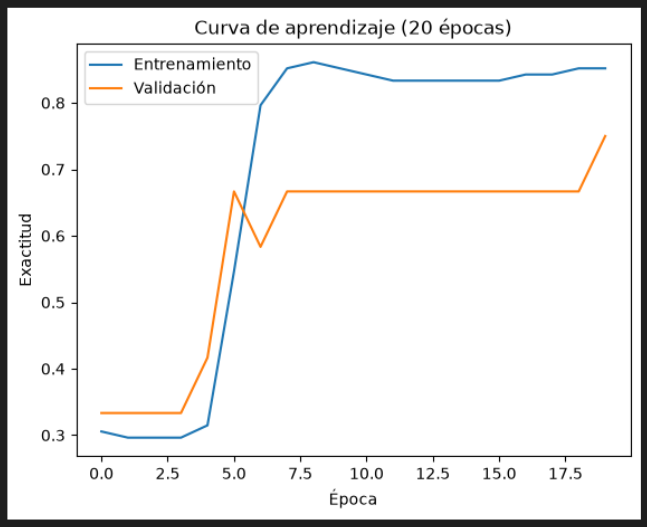
Layer (type)	Output Shape	Param #
dense (Dense)	(None, 10)	50
dense_1 (Dense)	(None, 8)	88
dense_2 (Dense)	(None, 3)	27

```
Total params: 165 (660.00 B)

Trainable params: 165 (660.00 B)

Non-trainable params: 0 (0.00 B)
```

```
Exactitud final en entrenamiento: 0.852
Exactitud en prueba: 0.767
Probabilidades: [0.975 0.011 0.014]
Especie predicha: setosa
Modelo guardado en modelo_iris_epochs20.keras
```



6. Reto: escribe un script 06_cargar_modelo.py que cargue modelo_iris.keras y prediga la especie de la flor [6.7, 3.0, 5.2, 2.3]. (Recuerda que también debes escalar los datos.)

E4_propuesto6.ipynb

IA_Guia04_TensorFlow_Keras > Ejercicios_propuestos4 > E4_propuesto6.ipynb > # 06_cargar_modelo.py / Celda para Jupyter Notebook (E4_propuesto6.ipynb) Jupyter Notebook

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | Python 3.12.10

```python
# 06_cargar_modelo.py / Celda para Jupyter Notebook (E4_propuesto6.ipynb)
import numpy as np
import keras
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler

# 1. Cargar el modelo usando la ruta relativa hacia atrás ("../")
# para ubicarlo en la carpeta superior IA_Guia04_TensorFlow_Keras
modelo = keras.models.load_model("../modelo_iris.keras")
print("Modelo cargado exitosamente desde la carpeta superior.")

# 2. Reconstruir el escalador utilizando los datos originales de Iris
# (Es obligatorio escalar la nueva flor exactamente igual que en el entrenamiento)
iris = load_iris()
X = iris.data
escalador = StandardScaler()
escalador.fit(X)

# 3. Definir y escalar la nueva flor: [6.7, 3.0, 5.2, 2.3]
flor_nueva = np.array([[6.7, 3.0, 5.2, 2.3]])
flor_escalada = escalador.transform(flor_nueva)

# 4. Hacer la predicción con el modelo
probabilidades = modelo.predict(flor_escalada, verbose=0)[0]
especie_predicha = iris.target_names[np.argmax(probabilidades)]

# 5. Mostrar los resultados
print("\n--- RESULTADO DE LA PREDICCIÓN ---")
print("Probabilidades por especie:")
for especie, prob in zip(iris.target_names, probabilidades):
    print(f" - {especie}: {prob * 100:.2f}%")

print(f"\nLa especie predicha para la flor [6.7, 3.0, 5.2, 2.3] es: **{especie_predicha}**")
```

[1] ✓ 2.3s Python

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are instal
Modelo cargado exitosamente desde la carpeta superior.

--- RESULTADO DE LA PREDICCIÓN ---
Probabilidades por especie:
 - setosa: 0.00%
 - versicolor: 0.54%
 - virginica: 99.46%

La especie predicha para la flor [6.7, 3.0, 5.2, 2.3] es: **virginica**
```